Smart Study

Feature & Architecture Guide

How the website works, how every feature was added,
and how the AI Assistant is built & wired up.

Offline-first student productivity app · HTML + CSS + vanilla JavaScript
No backend · No build tools · Data stays in the browser (localStorage)

Version 1.0 · Generated for the project owner
Repo: github.com/abhisheksinghcpj121/modern-to-do-list1

Contents

1. What Smart Study is
2. Tech stack & why it's static
3. Project file structure
4. How the app starts (boot sequence)
5. The data layer (storage + models)
6. Accounts & security (auth)
7. Feature-by-feature: how it works
8. Charts & analytics (dependency-free)
9. Global search (Ctrl+K)
10. The AI Assistant — how it was added
11. The AI Assistant — how it works
12. Privacy & security model
13. Running & deploying
14. Known limitations

> **Reading this guide:** every section explains a real part of the app. Code names (`js/ui.js` , `M.tasks` , `SS.ai`) match the actual files so you can open them and follow along.

1. What Smart Study is

Smart Study is a student productivity workspace that runs entirely in the browser. It combines task management, study planning, notes, goals, a Pomodoro focus timer, progress analytics and an AI study assistant — all in one place.

It was built as a **100% static site**: plain HTML, CSS and vanilla JavaScript. There is no server, no database, no framework and no build step. Every piece of user data (tasks, notes, sessions, goals, settings) is stored in the browser's `localStorage`, so the app works offline and nothing is uploaded anywhere unless you explicitly use an online AI provider.

Core principles the app follows

Principle	What it means in practice
Zero backend	Open a file or host the folder; it just works. No API server to run.
Data ownership	Your data lives on your device, keyed per account. Export/import gives you a full backup.
Safe rendering	All user content is drawn with `textContent`, never `innerHTML`, so a note can never inject markup or script.
Dependency-free	Charts are hand-drawn with CSS/SVG. No chart library, no CDN, no third-party scripts.
Graceful AI	The assistant works offline in Study mode, needs no key in Free mode, and supports bring-your-own-key providers.

Feature summary

Area	Highlights
Tasks	Priorities, subjects, due dates, overdue flags, recurring tasks, filters, search, completion undo via recurrence links.
Notes	Autosaving editor, tags, colours, pinning, instant search.
Goals	Deadlines, progress nudges, categories, streaks.
Study Plan	Weekly timetable, per-subject hour targets, exam countdowns.
Pomodoro	Focus/short/long cycles, custom durations, task linking, sound + notifications, session log.
Dashboard	Greeting, stats, completion ring, weekly focus chart, 4-week heatmap, donut, trend, best-hours, subject split.
Global search	`Ctrl+K` jumps to any task, note, goal, exam or plan slot.
AI Assistant	Study / Free / AI modes, streaming replies, safe markdown, multiple providers.
Settings	Profile, password, timer defaults, appearance, AI provider, backup/restore/wipe.

2. Tech stack & why it's static

Layer	Technology	Why
Markup	HTML5 (one file per screen)	Simple, hostable anywhere, easy to read.
Styling	Plain CSS with custom-property tokens	One theme system drives light & dark; no preprocessor.
Logic	Vanilla JavaScript (ES5-style, IIFEs)	No framework, no bundler; loads instantly and runs from `file://`.
Persistence	`localStorage`	Works offline, survives reloads, keyed per user.
Charts	CSS gradients + inline SVG	No chart library; keeps the app dependency-free.
AI	`fetch` + Server-Sent Events (SSE)	Streams replies token-by-token from the chosen provider.

> **Result:** the whole app is a folder of files. You can open `index.html` directly, host it on GitHub Pages, Netlify, or any static server — no install, no build, no server code.

Global namespacing

Every module attaches itself to a single global object, `window.SS`. For example `SS.storage`, `SS.models`, `SS.ui`, `SS.auth`, `SS.ai`. This keeps the code organised without modules or a bundler.

```
window.SS = window.SS || {};
// each file adds its own slice:
SS.ui = { toast, el, initChrome, search, ... };
SS.models = { tasks, notes, goals, plan, exams, subjects, sessions, ... };
```

3. Project file structure

```
smart study/
├── index.html            Landing page (login/signup, live hero stats)
├── login.html            Login screen
├── signup.html           Sign-up screen
├── dashboard.html        Overview, analytics, quick add + pomodoro timer
├── tasks.html            Task list with filters and search
├── notes.html            Notes library + editor
├── goals.html            Goal tracking
├── study-plan.html       Weekly timetable + exam countdowns + charts
├── settings.html         Profile, theme, backups, AI key
├── assistant.html        AI study assistant chat
├── preview.html          Dev-only visual map of every screen (noindex)
├── README.md             Project documentation
├── css/
│   ├── style.css         Design tokens, core components, motion
│   └── pages.css         Page-specific layouts
└── js/
    ├── storage.js        Namespaced localStorage wrapper + export/import
    ├── models.js         Shared CRUD layer for every collection
    ├── ui.js             DOM helpers, toasts, modals, theme, dates, global search
    ├── auth.js           Accounts, sessions, hashing, guest mode, guard
    ├── app.js            Shared page chrome (navbar, guard, theme)
    ├── ai.js             AI engine: providers, streaming, retrieval, markdown
    ├── assistant.js      Assistant page wiring (chat, quick actions)
    ├── settings.js       Settings page (profile, timer, AI, data)
    ├── tasks.js          Tasks page logic
    ├── notes.js          Notes page logic
    ├── goals.js          Goals page logic
    ├── studyplan.js      Study plan page logic
    ├── dashboard.js      Dashboard aggregation + stats
    ├── charts.js         Hand-built SVG/CSS charts
    ├── pomodoro.js       Focus timer + session log
    ├── landing.js        Landing hero behaviour
    └── authpage.js       Login/signup form logic
```

4. How the app starts (boot sequence)

Every signed-in screen is an HTML page that loads the same core scripts in the **exact same order**. Order matters because each layer depends on the one above it.

```
<script src="js/storage.js"></script>   <!-- 1. persistence       -->
<script src="js/models.js"></script>    <!-- 2. CRUD collections  -->
<script src="js/ui.js"></script>        <!-- 3. UI kit            -->
<script src="js/auth.js"></script>      <!-- 4. sessions / guard  -->
<script src="js/ai.js"></script>        <!-- 5. AI engine         -->
<script src="js/app.js"></script>       <!-- 6. shared chrome     -->
<script src="js/<page>.js"></script>    <!-- 7. this page's logic -->
```

Step by step

1. **storage.js** is defined first — it's the only layer that touches `localStorage`.
2. **models.js** builds collection objects (`SS.models.tasks`, etc.) on top of storage.
3. **ui.js** exposes DOM helpers, toasts, modals, theme control, date utilities and the global search engine.
4. **auth.js** manages accounts and the current session.
5. **ai.js** sets up the AI providers and the chat engine.
6. **app.js** runs on `DOMContentLoaded`: if nobody is signed in it bounces to the login page; otherwise it calls `SS.ui.initChrome()` which wires the navbar, theme, logout and the global search box.
7. **the page module** (e.g. `tasks.js`) also waits for `DOMContentLoaded`, calls `SS.auth.require()` as a second guard, then renders its own screen.

> **Two-layer guard:** `app.js` protects the shared chrome, and each page module calls `SS.auth.require()` first. An anonymous visitor can never see protected content.

5. The data layer

5.1 storage.js — the persistence wrapper

All reads/writes go through one small wrapper so keys stay namespaced and consistent. Data is stored **per user**:

```
smartStudy:session                → who is signed in
smartStudy:<email>:tasks          → that user's tasks
smartStudy:<email>:notes          → that user's notes
smartStudy:<email>:sessions       → focus sessions
smartStudy:<email>:ai             → AI config (provider, key)  ← never exported
```

A one-time migration also renames the app's former `studySync:` prefix to `smartStudy:` so old data is never lost.

Export / import

The whole workspace can be written to a JSON file and restored later. A defined `EXPORT_SKIP = ["ai"]` list means the **AI key is deliberately left out of backups**, and `Store.wipe()` clears everything including the AI config.

5.2 models.js — shared CRUD for every collection

Every collection is created by one factory, so all of them behave identically:

```
function make(name) {
  return {
    all(), save(list), add(item), find(id), update(id, patch), remove(id), clear()
  };
}

var M = {
  tasks: make("tasks"),   notes: make("notes"),   goals: make("goals"),
  sessions: make("sessions"), exams: make("exams"),
  subjects: make("subjects"), plan: make("plan"),
  study: { /* derived stats: minutesOn, todayMinutes, streak, focusSessions */ },
  taskView: { /* isRecurring, overdue, nextDate */ },
  completeTask(id) { /* marks done and spawns the next occurrence */ }
};
```

Recurring tasks (an important detail)

When you complete a repeating task, the finished copy is kept as history and a **new task is scheduled for the next matching date**. The link between them (`spawnedId`) is remembered so that re-opening the completed task can undo the newly created one.

```
completeTask(id):
  next = taskView.nextDate(task)
  if next: spawned = tasks.add({ ...same fields, date: next, completed: false })
  tasks.update(id, { completed: true, completedAt: now, spawnedId: spawned?.id })
```

6. Accounts & security (auth.js)

Capability	How it works
Sign up / login	Accounts stored locally with a hashed password. The session is a small record in `smartStudy:session`.
Guest mode	Jump in without an account; a guest profile is created on the spot.
Route guard	`SS.auth.require()` and `app.js` redirect anonymous visitors to `login.html?next=<page>`.
Safe `next`	`nextUrl()` only accepts local page names, so an attacker can't redirect to an external URL.
Per-user data	Collections are namespaced by email — switching accounts switches the whole workspace.

> **Honest limitation:** this is a client-side demo auth. Hashing deters casual snooping but is **not** equivalent to a real server-side authentication system. Don't store sensitive real passwords.

7. Feature-by-feature: how it works

7.1 Tasks `tasks.js`

- **Fields:** title, subject, date, priority (low/medium/high), notes, repeat, completed.
- **Views:** filter by subject, completion, overdue and recurrence; live search.
- **Overdue logic:** a pending task whose date is in the past is flagged as late.
- **Recurrence:** daily/weekly/custom — completing it spawns the next copy (see §5.2).
- **Stats:** total, done, pending, overdue and completion percentage.

7.2 Notes `notes.js`

- **Editor** autosaves as you type; no "Save" button needed.
- **Organisation:** tags, colour accents and pinning.
- **Search** filters the library instantly by title, body and tags.

7.3 Goals `goals.js`

- Each goal has a title, category, deadline and a progress percentage.
- Progress can be nudged up/down; completed goals are visually distinguished.
- Nearest-deadline-first ordering on the dashboard.

7.4 Study Plan `studyplan.js`

- **Weekly timetable:** click a session to edit; hover a day column to add a slot.
- **Subject hour targets:** planned hours compared against a weekly goal.
- **Exam countdowns:** days remaining, nearest first.

7.5 Pomodoro `pomodoro.js`

- Focus / short break / long break cycles with custom lengths.
- A long break triggers after a configurable number of rounds.
- You can link the timer to a task, so focus time is attributed to it.
- End-of-session **sound** (Web Audio beep) and opt-in **desktop notifications**.
- Each completed session is appended to the `sessions` log — the raw material for all the charts.
- Shortcut: press `Space` to start/pause.

7.6 Dashboard `dashboard.js`

Aggregates everything into one view:

- Time-aware greeting + date, and a daily-focus progress ring.
- Stat cards: pending, completed, focus-today, study streak.
- Today's tasks, goals snapshot, upcoming exams, task-status donut.
- Charts: weekly focus bars, a 4-week activity heatmap, a focus trend (7/14/30 days), a "best hours" grid and a subject split.
- Quick-add bar to drop a task for today without leaving the page.

7.7 Settings `settings.js`

- Profile info and (for registered accounts) password change.
- Focus timer defaults, daily focus goal.
- Appearance / theme.
- AI Assistant provider + key.
- Your data: export, restore and wipe.

7.8 Theme & motion `ui.js`

A single set of CSS custom properties (`--brand` , `--surface` , `--text` , `--border` , ...) defines both light and dark themes. Toggling the moon button flips a `data-theme` attribute, and every component re-colours automatically. A "reveal" motion system animates items into view and respects `prefers-reduced-motion` .

8. Charts & analytics (dependency-free)

There is no chart library. `charts.js` builds every visual by hand using CSS and inline SVG:

Chart	Built from	Shows
Progress ring / bar	conic-gradient + divs	Task completion %
Weekly focus bars	flex divs with heights	Minutes per day this week
4-week heatmap	CSS grid of cells	Activity vs daily goal
Focus trend	SVG polyline	7 / 14 / 30-day minutes
Best hours	CSS grid	Which hours you focus most
Status donut	conic-gradient	Done vs pending vs late
Subject split	stacked bars	Share of study time per subject

Because the visuals are pure CSS/SVG, they adapt to the theme automatically and add zero kilobytes of third-party code.

9. Global search (Ctrl+K) newest feature

A search box is injected into the navbar's top-right cluster by `initChrome()`, so it appears on **every signed-in page** without editing each HTML file. It searches across all collections at once and jumps straight to the matching page.

How the ranking works

`SS.ui.search(query, limit)` is a pure function that scores every item:

- An **exact phrase** match scores higher than a single-word match.
- Field weighting: **title > subject/tags > body**.
- Results are sorted by score and capped (default 8).

```
score(text):  +4 if the whole query appears,  +2 per matching word
task    = title + subject*0.6 + notes*0.35
note    = title + content*0.5 + tags*0.5
goal    = title + category*0.6
exam    = title + subject*0.6
plan    = subject + topic*0.6
subject= name
```

Interaction

- **Ctrl+K / ⌘K** focuses the box from anywhere.
- **↑ / ↓** move the highlighted result; **Enter** opens it; **Esc** closes.
- Typing is debounced (120 ms); an empty query hides the dropdown.
- A friendly "No matches for '…'" row appears for dead-end queries.

> This feature was added in `js/ui.js` (`search` + `initGlobalSearch`) plus a small CSS block, and covered by both the data-layer suite (ranking) and the jsdom suite (injection, keyboard, jump).

10. The AI Assistant — how it was added

The assistant is intentionally **pluggable**. Rather than hard-coding one AI service, the app defines a small adapter interface so any provider can be dropped in, and a "no key at all" provider works out of the box.

10.1 The three modes

Mode	Needs a key?	What powers it	Privacy note
Study	No	Built-in offline logic + your own data (no network).	100% on-device.
Free default	No	Public keyless endpoint (Pollinations `text.pollinations.ai`).	Messages go to a third party — disclosed in the UI.
AI	Yes (your key)	OpenAI-compatible, Anthropic or Gemini.	Data goes only to the provider you choose.

10.2 The adapter pattern

`js/ai.js` holds a `PROVIDERS` list. Each entry describes how to talk to that service: its endpoint, how to build the request, and how to read the streamed reply.

```
PROVIDERS = [
  { id: "free",     keyless: true, needsBase: true,  ... },  // Pollinations
  { id: "openai",    ... },   // OpenAI-compatible endpoints
  { id: "anthropic",... },   // Anthropic Messages API
  { id: "gemini",    ... }    // Google Gemini
]
```

Shared helpers keep the page code simple and provider-agnostic:

Helper	Purpose
`providerNeedsKey(id)`	True unless the provider is keyless.
`isKeyless(id)`	Used by the settings UI to hide the key field.
`hasKey()`	Returns true for keyless providers too.
`activeProvider()`	Resolves the configured provider.
`ask(...)`	Sends the request and streams the answer back.
`parseSSE(...)`	Pure function that reconstructs events split across chunks.

10.3 Config & storage

AI settings live under the store key `ai` (provider, optional key). Crucially, this key is in the export-skip list, so a backup never contains your API key. The default provider is `free`, which means the assistant starts working instantly with zero configuration.

10.4 How the pieces connect

File	Role
`js/ai.js`	The engine: providers, config, streaming, markdown, study-mode retrieval.
`js/assistant.js`	The chat page: message thread, quick actions, and the three-mode banner.
`js/settings.js`	Provider selector + key field; hides the key for keyless providers.

`settings.html`	The AI card markup.
`assistant.html`	The chat shell, banner and composer.

11. The AI Assistant — how it works

11.1 Sending a message

1. You type a question in the composer and send it.
2. `SS.ai.ask()` checks whether the active provider needs a key (`hasKey()` is `true` for keyless providers, so Free mode is never blocked).
3. The request is built in the provider's expected shape and sent with `fetch`.
4. The response arrives as a **stream**; the reply text is appended as it arrives, so you see it "type" itself.

What a Free request looks like

```
POST https://text.pollinations.ai/openai
Content-Type: application/json
(no Authorization header — it is keyless)

{
  "model": "openai",
  "stream": true,
  "messages": [ { "role": "user", "content": "Explain mitochondria" } ]
}
```

11.2 Streaming with parseSSE

Streamed replies arrive as Server-Sent Events. Network chunks don't respect event boundaries, so a buffered parser stitches them back together. It reads only the `delta.content` field and ignores the separate `delta.reasoning` channel, then stops at `[DONE]`.

```
data: {"choices":[{"delta":{"content":"The "}}]}
data: {"choices":[{"delta":{"content":"mitochondrion…"}}]}
data: {"choices":[{"delta":{"reasoning":"…ignored…"}}]}
data: [DONE]
```

> **Why a pure parser?** `parseSSE` takes text and returns events, with no network or DOM dependency — so it can be unit-tested with artificial chunk splits, which is exactly how the test suite verifies it.

11.3 Safe markdown rendering

The model may return markdown (lists, bold, code). The renderer converts a **safe subset** to DOM nodes, and every piece of text is applied with `textContent`. The regexes are compiled per call — a past bug where a shared/frozen regex broke repeated rendering was fixed this way. The result: AI output can **never** inject HTML or script into the page.

11.4 Study mode (offline)

In Study mode there is no network at all. The assistant answers from built-in logic and can retrieve from your own notes/tasks to construct a helpful reply — fully on-device and instant.

11.5 Privacy trade-off, stated plainly

> In **Free** mode your prompt (and any note you ask about) is sent to a public third-party endpoint. It is rate-limited and can be slow or briefly down. If that matters to you, switch to **Study** mode (offline) or to an **AI** provider with your own key, where data only reaches the provider you chose. The key is stored only in this browser and is excluded from every backup.

12. Privacy & security model

Concern	How the app handles it
API key	Never hard-coded in the client source. Stored in `localStorage`, sent only to the chosen provider's endpoint, and excluded from exports.
User content	Rendered with `textContent` everywhere; no `innerHTML` for user data.
Third-party scripts	None. No CDNs, no trackers, no analytics beacon.
Data location	Browser `localStorage`, namespaced per account.
Backups	Manual JSON export/import; the AI config is intentionally skipped.
AI data flow	Study = offline; Free = third party; AI = your provider only.

> **Note on the debug token:** the GitHub token used to publish this repo was shared in chat and should be revoked at `github.com/settings/tokens`. Publishing does not depend on it afterwards.

13. Running & deploying

Run locally

- **Simplest:** double-click `index.html` — it opens in any browser.
- **Recommended:** serve the folder with any static server (avoids some `file://` quirks), e.g. `python -m http.server` or the VS Code "Live Server" extension.

Deploy to GitHub Pages

1. Push the files to the repository's `main` branch.
2. Go to **Settings** → **Pages**.
3. Source: **Deploy from a branch** → `main` / `/ (root)` → **Save**.
4. Your site goes live at `https://<username>.github.io/<repo>/` within a minute.

> **This project is already live at:**
> `https://abhisheksinghcpj121.github.io/modern-to-do-list1/`

14. Known limitations

- **Client-side auth** is a demo — hashing is local, not server-grade.
- **Free AI** relies on a public, rate-limited endpoint; it can be slow or temporarily unavailable.
- **No cross-tab sync:** open tabs don't reflect each other's changes live.
- **No service worker:** the app is offline-friendly via `localStorage`, but not yet an installable PWA.
- **Data is per-device:** clearing browser storage removes it — keep regular JSON backups.

Possible next steps

Idea	What it adds
Undo delete	One-tap restore for tasks/notes/goals.
Session history page	Full focus-session log (data already recorded).
Sub-tasks	Break a task into checklist steps.
Reminders	Opt-in notifications for due tasks and exams.
Note export	Download a note as `.md` / `.txt`.
PWA / offline	Service worker + installability.
Ask-about-my-data AI	Give the assistant your plan/notes context for richer answers.